

Universidade Federal de Sergipe
Departamento de Computação
Docente: Rafael Oliveira Vasconcelos

Projeto Unidade 1

Sistemas Distribuídos – T01 2026.2

Discentes
Arthur Vinícius Costa Barreto
Bruno Carvalho Santana Rocha
Danilo Campos Deichmann
Filipe Ciríaco Marcelino do Nascimento
Sidnei Silva Gonçalves Junior

São Cristóvão – SE
21 de outubro de 2026

1 Introdução

O presente documento descreve a proposta técnica para o desenvolvimento de um Simulador de Bolsa de Valores Multiplayer, desenvolvido como requisito de avaliação da disciplina de Sistemas Distribuídos. O sistema permitirá a conexão simultânea de múltiplos investidores em um ambiente de mercado financeiro virtual, abordando desafios de consistência, concorrência e tolerância a falhas.

2 Regras do Mercado, Ativos e Investidores

2.1 Perfil dos Investidores e Capital Inicial

Cada jogador iniciará com um capital virtual de R$ 10.000,00 e uma carteira vazia de ativos. Para focar a complexidade nos desafios de sistemas distribuídos (como concorrência e sincronização), o mercado operará com corretagem zero (sem taxas por transação) e sem limites de quantidade por negociação, desde que haja saldo ou ativos suficientes.

2.2 Ativos Virtuais

O mercado disponibilizará 4 ativos reais baseados na B3, representando diferentes setores da economia:

- **PETR4:** Petrobras - Preço base: R$ 35,00
- **VALE3:** Vale - Preço base: R$ 68,00
- **ITUB4:** Itaú Unibanco - Preço base: R$ 30,50
- **MGLU3:** Magazine Luiza - Preço base: R$ 2,10

2.3 Mecânica de Negociação e Eventos Econômicos

As ordens seguirão o princípio de prioridade de preço e prioridade temporal no *Matching Engine*. O mercado funcionará 24 horas por dia (sem interrupções na simulação). Para estimular a volatilidade, o sistema gerará "Eventos Econômicos" aleatórios a cada 10 minutos (ex: "Boom Tecnológico" ou "umento de Juros", impactando os preços de referência dos ativos e incentivando os investidores conectados a adaptarem as suas estratégias.

3 Cronograma de Desenvolvimento

O cronograma abaixo reflete as atividades distribuídas até as entregas:

- **21/10/2026:** Entrega da Proposta, diagrama arquitetural, infraestrutura em contêineres e Protótipo funcional (login JWT, painel e envio em tempo real via STOMP).
- **Novembro/2026 (Semanas 1-2):** Integração completa do Matching Engine na memória com a base de dados relacional.
- **Novembro/2026 (Semanas 3-4):** Execução de testes de carga em larga escala, indução de falhas e refatoração arquitetural.
- **30/11/2026:** Apresentação final e entrega do código com documentação.

4 Arquitetura Distribuída e Tecnologias

4.1 Arquitetura Proposta e Diagrama

O sistema adota uma arquitetura modular em contêineres orquestrada via Docker Compose. O fluxo de comunicação baseia-se na interação do **Navegador (Home Broker)** servido por um contêiner **Nginx**, que roteia as requisições e a comunicação em tempo real para o **Backend Spring Boot (REST + STOMP)**. O Backend centraliza toda a lógica de negócio, a gestão de estado em memória para processamento rápido e persiste os dados consolidados num banco **PostgreSQL**.

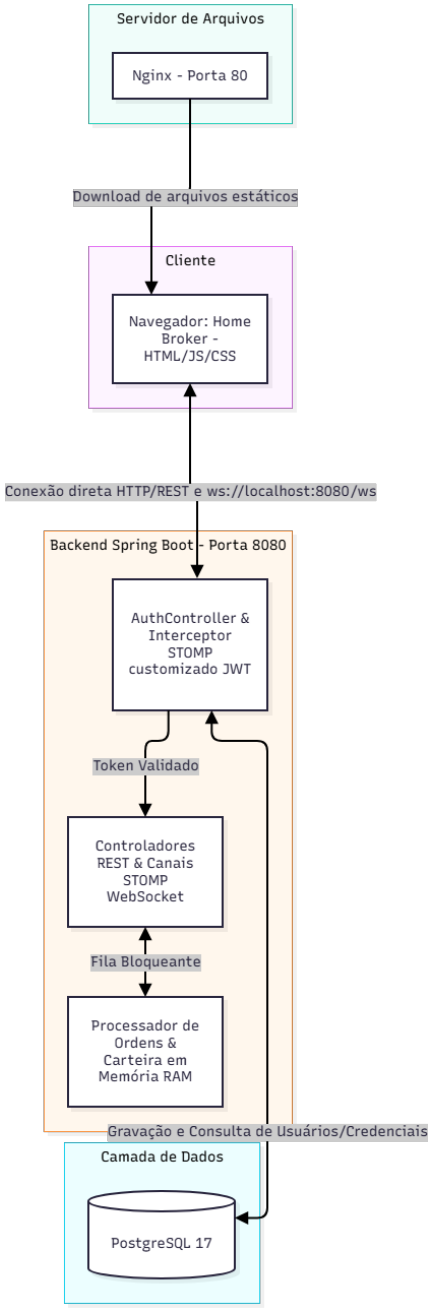


Figura 1: Diagrama de Componentes da Arquitetura do Sistema.

4.2 Tecnologias Escolhidas

A solução foi conteinerizada via Docker, atendendo aos requisitos mínimos. As ferramentas selecionadas são:

- **Backend (Servidor e Matching Engine):** Desenvolvido em Java 21 com Spring Boot 3, utilizando Spring WebSocket/STOMP, Spring Data JPA, Lombok e Maven para gerenciar concorrência e o ciclo de vida das ordens.
- **Frontend (Home Broker):** Desenvolvido em HTML5, CSS3 puro e JavaScript nativo (Vanilla JS), sem frameworks de terceiros, servido de forma estática por um contêiner Nginx na porta 80.
- **Banco de Dados e Cache:** O projeto utiliza PostgreSQL 17 na porta 5432 para persistência. O controle de cache e carteira do protótipo é feito de forma ultrarrápida em memória pelo próprio Java, através de estruturas concorrentes (`ConcurrentHashMap`), não exigindo o Redis.

4.3 Protocolos de Comunicação

O sistema utiliza WebSocket nativo em conjunto com o subprotocolo **STOMP** (em `ws://localhost:8080/ws`). Os canais implementados incluem:

- **Público (Broadcast):** O canal `/topic/market` para envio contínuo de eventos do mercado (cotações) a todos os clientes.
- **Privado Ponto-a-Ponto:** Canais como `/user/queue/orders` (para retornos de aceite/rejeição das ordens) e `/user/queue/replies` (para listagens de ordens do utilizador).

Todas as mensagens transitam através de um envelope padronizado (`MarketEvent<T>`), contendo metadados cruciais como `type`, `requestId`, `eventId`, `timestamp` e `data`. Operações pontuais e estáticas utilizam HTTP/REST tradicional.

5 Estratégias de Dados, Consistência e Falhas

5.1 Banco de Dados e Persistência

A persistência abrangerá utilizadores, ordens e histórico de negociações no PostgreSQL. O banco relacional foi justificado pela necessidade estrita de transações ACID na gestão dos ativos e capitais dos investidores a longo prazo.

5.2 Estratégia de Consistência e Concorrência

O controle de concorrência e consistência do protótipo foi implementado a nível de aplicação em Java, garantindo alta vazão sem corrupção de dados:

- **Idempotência e Ordenação:** O `OrderService` gera números sequenciais atómicos num bloco sincronizado e utiliza chaves `userId|clientOrderId` para evitar retransmissões acidentais da mesma ordem.

- **Fila Bloqueante Limitada:** O `OrderProcessor` gere as ordens através de uma `LinkedBlockingQueue(10000)` processada em *thread* dedicada. Caso a fila atinja o limite de capacidade, novas ordens recebem a rejeição segura de `SYSTEM_BUSY`.
- **Atomicidade de Carteira:** A carteira (`InMemoryWallet.java`) executa transações de reserva de saldo (para compras) e bloqueio de ações (para vendas) de forma nativamente sincronizada em memória.

Para a versão de produção (Parte 2), a persistência total da carteira utilizará travamento pessimista (*Pessimistic Locking* via `SELECT ... FOR UPDATE`) no banco de dados.

5.3 Tolerância a Falhas

Para lidar com quedas de conexão ou reinícios, o sistema prevê:

- **Autenticação *Stateless*:** O acesso e o reconhecimento da sessão utilizam tokens JWT e interceptação nativa de frames STOMP (`AuthChannelInterceptor.java`), permitindo que o investidor se reconecte sem perder o seu estado de sessão.
- **Prevenção contra Falhas Abruptas:** Em casos de encerramento forçado da aplicação, o sistema registra *logs* avisando sobre as ordens na fila que foram descartadas (`ORDER_QUEUE_DISCARDED`).

5.4 Observabilidade e Testes Automatizados

Em atendimento ao requisito R13, a implementação foca na qualidade de rastreamento do código:

- **Testes (JUnit 5):** A aplicação já conta com *test suites* consolidados, contemplando simulações de concorrência (`OrderConcurrencyTest`), limites de fila (`OrderProcessingTest`), integração STOMP (`OrderStompIntegrationTest`) e validação atómica da carteira.
- **Observabilidade de Logs:** O fluxo completo de cada ordem registra transições padronizadas nos *logs* (ex: `ORDER_RECEIVED` → `ORDER_ACCEPTED` ou `ORDER_REJECTED`), indicando sempre o motivo da falha (como `INSUFFICIENT_FUNDS` ou `INVALID_FIELDS`).

6 Cronograma de Desenvolvimento

O cronograma abaixo reflete as atividades distribuídas até as entregas:

- **21/10/2026:** Entrega da Proposta, diagrama arquitetural, infraestrutura em contêineres e Protótipo funcional (login JWT, painel e envio em tempo real via STOMP).
- **Novembro/2026 (Semanas 1-2):** Integração completa do Matching Engine na memória com a base de dados relacional e implementação dos NPCs.
- **Novembro/2026 (Semanas 3-4):** Execução de testes de carga em larga escala, indução de falhas e refatoração arquitetural.
- **30/11/2026:** Apresentação final e entrega do código com documentação.